

Lecture 6: Classical iterative methods

Jamie Haddock

Table of contents

1	Matrix structure (part 1)	1
1.1	Diagonal dominance	1
1.2	Symmetric positive definite matrices	1
2	Classical Iterative Methods for Solving Linear Systems	2
2.1	Jacobi's Method	2
2.2	Splitting method form of Jacobi update	3
2.3	Convergence of Jacobi Method	5
2.4	Gauss-Seidel method	5
2.5	Splitting method form of G-S update	6
2.6	Convergence of Gauss-Seidel Method	7
2.7	Successive over-relaxation (SOR)	8
2.8	Splitting method form of SOR update	8
2.9	Convergence of SOR	8
2.10	Kaczmarz method	9
2.11		9

1 Matrix structure (part 1)

Many matrices typically encountered in scientific computing have special structure. It can be *very* helpful to understand and exploit these special structures!

1.1 Diagonal dominance

An $n \times n$ matrix $\mathbf{A}$ is **strictly (row) diagonally dominant** if

$$|A_{ii}| > \sum_{j=1/j \neq i}^n |A_{ij}| \text{ for each } i = 1, \dots, n.$$

This says that the magnitude of entries on the diagonal are larger than the sum of magnitudes of entries in the same row off-diagonal.

- Diagonally dominant matrices are guaranteed to be invertible.
- Diagonally dominant matrices do not need row-pivoting for elimination/LU stability.

1.2 Symmetric positive definite matrices

Suppose $\mathbf{A} \in \mathbb{R}^{n \times n}$ and $\mathbf{x} \in \mathbb{R}^n$. Note that $\mathbf{x}^\top \mathbf{A} \mathbf{x}$ is scalar valued – this is called a **quadratic form**.

Definition: Symmetric positive (semi-)definite matrix

A real $n \times n$ matrix $\mathbf{A}$ is called a **symmetric positive definite matrix** (SPD) if it is symmetric and, for all $\mathbf{x} \neq 0$,

$$\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0.$$

A matrix is a **symmetric positive semi-definite matrix** if the inequality above holds but possibly with equality for some nonzero vectors $\mathbf{x}$.

This definition is equivalent to assuming that $\mathbf{A} \in \mathbb{R}^n$ has:

1. all eigenvalues strictly positive (nonnegative), or
2. all leading principal minors are strictly positive (nonnegative).

2 Classical Iterative Methods for Solving Linear Systems

Today we'll see some classical iterative methods for solving linear systems. Several of these are particular examples of **splitting methods**.

2.1 Jacobi's Method

The first method hinges on the observation that if $\mathbf{A}\mathbf{x} = \mathbf{b}$ then

$$A_{i1}x_1 + A_{i2}x_2 + \cdots + A_{ii}x_i + \cdots + A_{in}x_n = b_i \text{ for } i = 1, \dots, n.$$

This can be arranged as

$$x_i = \frac{1}{A_{ii}} \left[\sum_{j=1, j \neq i}^n (-A_{ij}x_j) + b_i \right] \text{ for } i = 1, \dots, n.$$

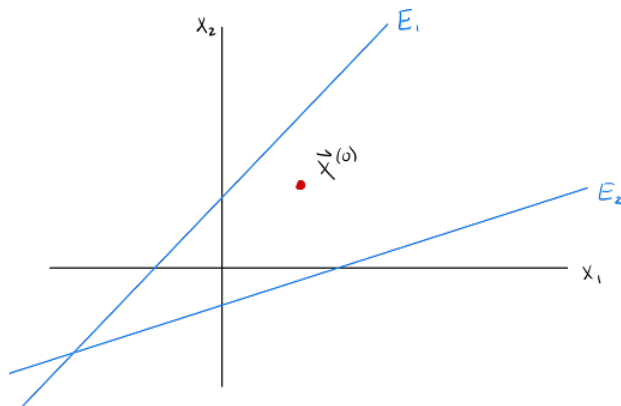
Jacobi's method uses this observation iteratively, forcing the i th component of the current approximation to satisfy the i th equation, that is $\mathbf{x}^{(k)}$ satisfies

$$x_i^{(k)} = \frac{1}{A_{ii}} \left[\sum_{j=1, j \neq i}^n (-A_{ij}x_j^{(k-1)}) + b_i \right] \text{ for } i = 1, \dots, n.$$

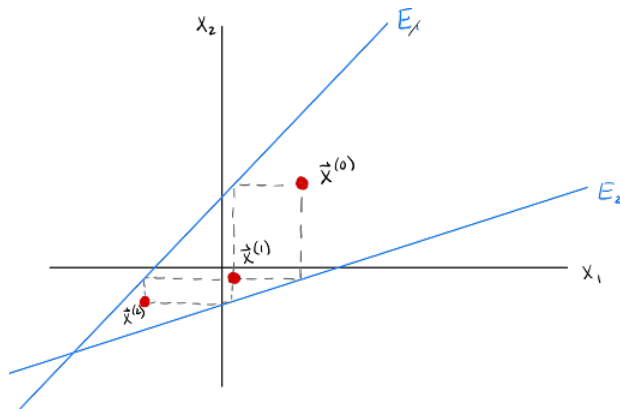
Exercise: Sketch Jacobi's method

Sketch two iterations of Jacobi's method applied to the system visualized below with the given initial iterate, $\mathbf{x}^{(0)}$.

System:



Answer:



You should also think about what happens here if the order of the equations is changed or if the initial iterate is somewhere else. Spoiler alert: Jacobi's method doesn't always converge!

2.2 Splitting method form of Jacobi update

We can collect all of the updates to the coordinates of $\mathbf{x}^{(k)}$ together into a vector update.

Note that we can rewrite $\mathbf{Ax} = \mathbf{b}$ as

$$\mathbf{D}(\mathbf{A})\mathbf{x} + \mathbf{L}(\mathbf{A})\mathbf{x} + \mathbf{U}(\mathbf{A})\mathbf{x} = \mathbf{b}$$

where $\mathbf{D}(\mathbf{A})$ is the diagonal matrix containing the diagonal elements of $\mathbf{A}$, $\mathbf{L}(\mathbf{A})$ is the strictly lower-triangular matrix containing the strictly lower-triangular elements of $\mathbf{A}$, and $\mathbf{U}(\mathbf{A})$ is the strictly upper-triangular matrix containing the strictly upper-triangular elements of $\mathbf{A}$.

In this notation, the Jacobi iterates satisfy

$$\mathbf{x}^{(k)} = -\mathbf{D}(\mathbf{A})^{-1}[\mathbf{L}(\mathbf{A}) + \mathbf{U}(\mathbf{A})]\mathbf{x}^{(k-1)} + \mathbf{D}(\mathbf{A})^{-1}\mathbf{b}.$$

```
%%julia
using LinearAlgebra
using Plots
```

```
%%julia
"""
    jacobi(A,b,x_0,N)

Run N iterations of Jacobi's method on system Ax=b with initial iterate x_0.
"""
function jacobi(A,b,x_0,N,x_true)
    D = Diagonal(A);
    L = tril(A,-1);
    U = triu(A,1);

    x_old = x_0
    errs = zeros(N+1)
    errs[1] = norm(x_old - x_true)/norm(x_true)
    for i in 1:N
        x_new = D\(-(L+U)*x_old + b);
        errs[i+1] = norm(x_new - x_true)/norm(x_true);
    end
end
```

```

        x_old = x_new;
    end
    return x, errs
end

```

jacobi

Now, we'll run a small experiment to check that our code works!

```

%%julia
n = 100
#A = randn(n,n)
A = diagm(ones(n)) + 0.05*randn(n,n)
x_true = ones(n)
b = A*x_true;
x = zeros(n);

```

```

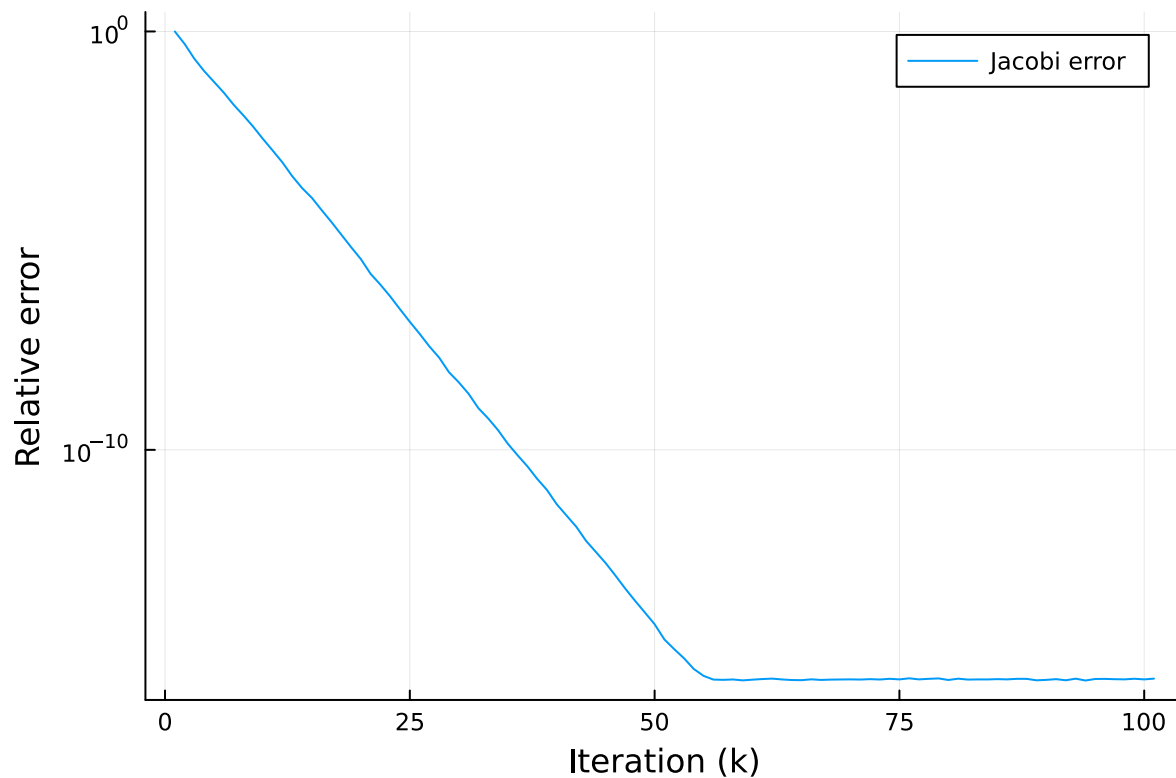
%%julia
N = 100
x, errs = jacobi(A,b,zeros(n),N,x_true);

```

```

%%julia
plot(1:N+1,errs,yscale=:log10,label="Jacobi error",xaxis="Iteration (k)", yaxis="Relative error")

```



2.3 Convergence of Jacobi Method

Theorem: Jacobi convergence

If $\mathbf{A}$ is strictly diagonally dominant, then Jacobi's method converges for any initial guess $\mathbf{x}^{(0)}$.

To prove this theorem, we need Gershgorin's disc theorem.

Theorem: Gershgorin's disc theorem

All eigenvalues of the matrix $\mathbf{M}$ lie in $\cup_{i=1}^n \mathcal{D}_i$ where $\mathcal{D}_i := \{z \in \mathbb{C} : |z - \mathbf{M}_{ii}| \leq \sum_{j \neq i} |\mathbf{M}_{ij}|\}$.

Proof of Jacobi convergence:

Note that the diagonal entries of $-\mathbf{D}(\mathbf{A})^{-1}[\mathbf{L}(\mathbf{A}) + \mathbf{U}(\mathbf{A})]$ are zero, and the off diagonal entries are $|\mathbf{A}_{ij}|/|\mathbf{A}_{ii}|$. Thus, Gershgorin's disc theorem says that all eigenvalues of $-\mathbf{D}(\mathbf{A})^{-1}[\mathbf{L}(\mathbf{A}) + \mathbf{U}(\mathbf{A})]$ have magnitude less than or equal to $\sum_{j \neq i} |\mathbf{A}_{ij}|/|\mathbf{A}_{ii}| < 1$. Thus, $\rho(-\mathbf{D}(\mathbf{A})^{-1}[\mathbf{L}(\mathbf{A}) + \mathbf{U}(\mathbf{A})]) < 1$, and we have that Jacobi's method converges.

2.4 Gauss-Seidel method

We can possibly improve Jacobi's method by taking a second look at the equation

$$x_i^{(k)} = \frac{1}{A_{ii}} \left[\sum_{j=1, j \neq i}^n (-A_{ij}x_j^{(k-1)}) + b_i \right] \text{ for } i = 1, \dots, n.$$

If we think about doing the updates to the entries of $\mathbf{x}^{(k)}$ sequentially, we see that we update $x_1^{(k)}$ before updating $x_2^{(k)}$, but in the update of $x_2^{(k)}$, we use $x_1^{(k-1)}$, despite having the (likely better) estimate for the first entry computed in $x_1^{(k)}$.

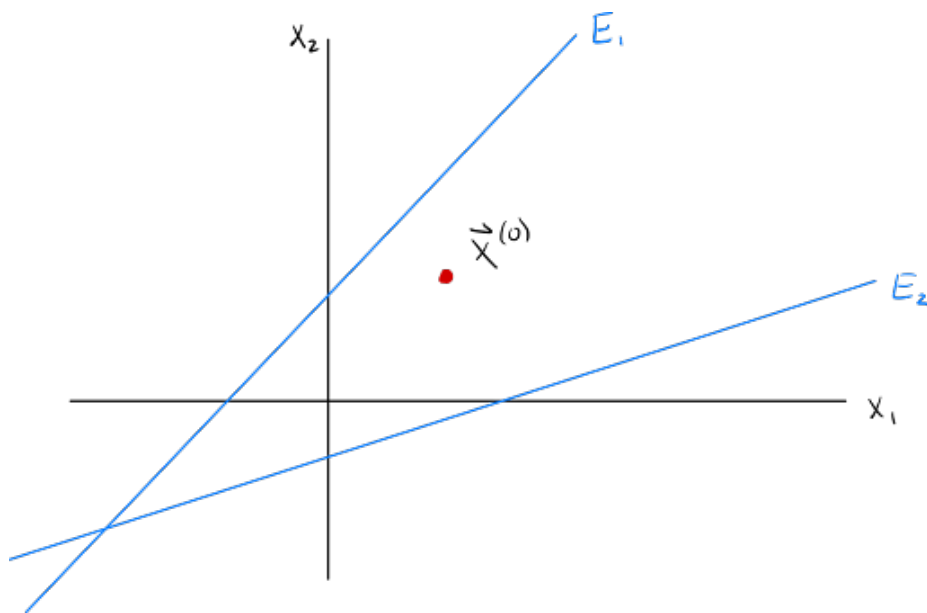
This motivates using the most up-to-date estimates available in this update:

$$x_i^{(k)} = \frac{1}{A_{ii}} \left[\sum_{j=1}^{i-1} (-A_{ij}x_j^{(k)}) + \sum_{j=i+1}^n (-A_{ij}x_j^{(k-1)}) + b_i \right] \text{ for } i = 1, \dots, n.$$

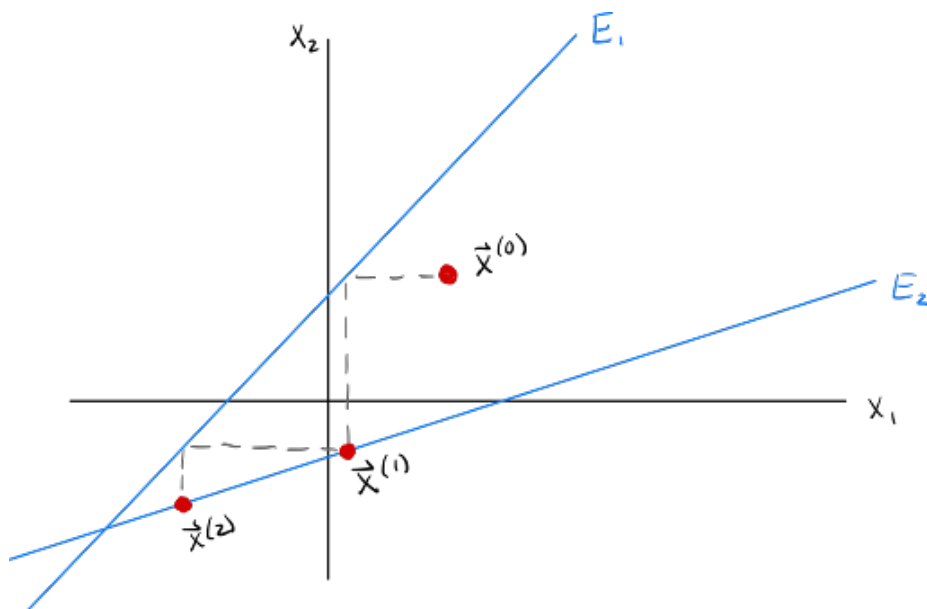
Exercise: Sketch the Gauss-Seidel method

Sketch two iterations of the Gauss-Seidel method applied to the system visualized below with the given initial iterate, $\mathbf{x}^{(0)}$.

System:



Answer:



2.5 Splitting method form of G-S update

In the same way we did for Jacobi, we can succinctly write the Gauss-Seidel update in matrix-vector form.

Noting, now, that the update

$$x_i^{(k)} = \frac{1}{A_{ii}} \left[\sum_{j=1}^{i-1} (-A_{ij}x_j^{(k)}) + \sum_{j=i+1}^n (-A_{ij}x_j^{(k-1)}) + b_i \right] \text{ for } i = 1, \dots, n$$

involves the entire lower triangular part of $\mathbf{A}$ interacting with entries of $\mathbf{x}^{(k)}$,

we can rewrite this as

$$[\mathbf{D}(\mathbf{A}) + \mathbf{L}(\mathbf{A})]\mathbf{x}^{(k)} + \mathbf{U}(\mathbf{A})\mathbf{x}^{(k-1)} = \mathbf{b}.$$

Thus, the update is

$$\mathbf{x}^{(k)} = -[\mathbf{D}(\mathbf{A}) + \mathbf{L}(\mathbf{A})]^{-1}\mathbf{U}(\mathbf{A})\mathbf{x}^{(k-1)} + [\mathbf{D}(\mathbf{A}) + \mathbf{L}(\mathbf{A})]^{-1}\mathbf{b}.$$

2.6 Convergence of Gauss-Seidel Method

Theorem: Gauss-Seidel convergence

If $\mathbf{A}$ is symmetric positive definite, then the Gauss-Seidel method converges for any initial guess $\mathbf{x}^{(0)}$.

To prove this theorem, we need the Householder-John theorem.

Theorem: Householder-John theorem

If the real matrices $\mathbf{A}$ and $\mathbf{A} - \mathbf{B} - \mathbf{B}^\top$ are both symmetric positive definite, and $\mathbf{B}$ is real, then the spectral radius of

$$\mathbf{H} = (\mathbf{A} - \mathbf{B})^{-1}\mathbf{B}$$

satisfies $\rho(\mathbf{H}) < 1$.

Proof of Gauss-Seidel convergence:

If $\mathbf{A}$ is symmetric, then $\mathbf{L}(\mathbf{A}) = \mathbf{U}(\mathbf{A})^\top$. In the Householder-John theorem, choose $\mathbf{A} = \mathbf{D}(\mathbf{A}) + \mathbf{L}(\mathbf{A}) + \mathbf{U}(\mathbf{A})$, and $\mathbf{B} = \mathbf{U}(\mathbf{A}) = \mathbf{L}(\mathbf{A})^\top$. Now, by assumption $\mathbf{A}$ is symmetric positive definite, and thus, so must $\mathbf{A} - \mathbf{B} - \mathbf{B}^\top = \mathbf{D}(\mathbf{A})$ be. (Why?) Thus, by the Householder-John theorem, we have that $\rho(\mathbf{H}) < 1$ for $\mathbf{H} = (\mathbf{A} - \mathbf{B})^{-1}\mathbf{B} = [\mathbf{D}(\mathbf{A}) + \mathbf{L}(\mathbf{A})]^{-1}\mathbf{U}(\mathbf{A})$, and the Gauss-Seidel method must converge.

Exercise: Implement the Gauss-Seidel method

Complete the Gauss-Seidel method code below and run this method on the same experimental system above.

```
%%julia
"""
    gaussseidel(A,b,x_0,N)

Run N iterations of the Gauss-Seidel method on system Ax=b with initial iterate x_0.
"""
function gaussseidel(A,b,x_0,N,x_true)
    D = Diagonal(A);
    L = tril(A,-1);
    U = triu(A,1);

    x_old = x_0
    errs = zeros(N+1)
    errs[1] = norm(x_old - x_true)/norm(x_true)
    for i in 1:N
        x_new = ##### complete this code!
        errs[i+1] = norm(x_new - x_true)/norm(x_true);
        x_old = x_new;
    end
    return x, errs
end
```

gaussseidel

2.7 Successive over-relaxation (SOR)

Gauss-Seidel already improves on Jacobi by using each coordinate's most up-to-date value as soon as it's available. A natural next question: once we have the Gauss-Seidel update, could we do even better by moving *further* in that direction — or, if Gauss-Seidel overshoots, by damping the step?

Write the Gauss-Seidel value for coordinate i as

$$x_i^{\text{GS}} = \frac{1}{A_{ii}} \left[\sum_{j=1}^{i-1} (-A_{ij}x_j^{(k)}) + \sum_{j=i+1}^n (-A_{ij}x_j^{(k-1)}) + b_i \right].$$

The **successive over-relaxation (SOR)** method introduces a relaxation parameter $\omega > 0$ and takes the new iterate to be a weighted average of the old value $x_i^{(k-1)}$ and the Gauss-Seidel value x_i^{GS} :

$$x_i^{(k)} = (1 - \omega)x_i^{(k-1)} + \omega x_i^{\text{GS}}.$$

Substituting the Gauss-Seidel expression gives the SOR update

$$x_i^{(k)} = (1 - \omega)x_i^{(k-1)} + \frac{\omega}{A_{ii}} \left[\sum_{j=1}^{i-1} (-A_{ij}x_j^{(k)}) + \sum_{j=i+1}^n (-A_{ij}x_j^{(k-1)}) + b_i \right] \text{ for } i = 1, \dots, n.$$

Notice that $\omega = 1$ recovers Gauss-Seidel exactly. Taking $\omega > 1$ is called **over-relaxation** (we push past the Gauss-Seidel point, hoping to accelerate convergence), while $0 < \omega < 1$ is **under-relaxation** (we damp the step, which can help stabilize an otherwise oscillatory iteration).

2.8 Splitting method form of SOR update

As with Jacobi and Gauss-Seidel, we can collect the SOR update into matrix-vector form.

Multiplying through by A_{ii} and separating known $(k-1)$ from unknown (k) terms, the update becomes

$$[\mathbf{D}(\mathbf{A}) + \omega \mathbf{L}(\mathbf{A})]\mathbf{x}^{(k)} = [(1 - \omega)\mathbf{D}(\mathbf{A}) - \omega \mathbf{U}(\mathbf{A})]\mathbf{x}^{(k-1)} + \omega \mathbf{b}.$$

Thus, the update is

$$\mathbf{x}^{(k)} = [\mathbf{D}(\mathbf{A}) + \omega \mathbf{L}(\mathbf{A})]^{-1} [(1 - \omega)\mathbf{D}(\mathbf{A}) - \omega \mathbf{U}(\mathbf{A})]\mathbf{x}^{(k-1)} + \omega [\mathbf{D}(\mathbf{A}) + \omega \mathbf{L}(\mathbf{A})]^{-1} \mathbf{b}.$$

When $\omega = 1$ this is exactly the Gauss-Seidel splitting from before.

2.9 Convergence of SOR

Theorem: SOR convergence

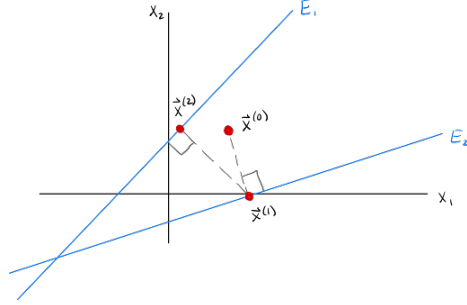
If $\mathbf{A}$ is symmetric positive definite, then the SOR method converges for any initial guess $\mathbf{x}^{(0)}$ if and only if $0 < \omega < 2$.

In order to optimize performance, one would hope to choose the optimal value ω . This is difficult, but in certain situations one can exactly evaluate the spectral radius of the SOR update matrix in terms of the spectral radius of the Jacobi update matrix, and thus choose the optimal ω .

2.10 Kaczmarz method

The Jacobi and Gauss-Seidel methods update using individual coordinate direction updates onto the hyperplanes defined by the individual equations. This is not the only way that the iterates can move towards these hyperplanes!

Perhaps the most natural updating strategy is to project each iterate onto a hyperplane to produce the subsequent iterate. This strategy is known as the *Kaczmarz method*. Note that this method is not a splitting method – it is a different, and common type of method – a **projection method**!



The update for this type of method is given by $\mathbf{x}^{(k)} = \mathbf{x}^{(k-1)} + \frac{b_i - \mathbf{a}_i^T \mathbf{x}^{(k-1)}}{\|\mathbf{a}_i\|^2} \mathbf{a}_i$ where $\mathbf{a}_i$ is the i th row of $\mathbf{A}$ and i is chosen from $1, \dots, n$.

2.11

Exercise: Decreasing error

If $\mathbf{x}^*$ satisfies $\mathbf{A}\mathbf{x}^* = \mathbf{b}$, then

$$\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_2 \leq \|\mathbf{x}^{(k-1)} - \mathbf{x}^*\|_2.$$

Show that this is true!

Answer:

Note that $\mathbf{x}^*$ lies in the intersection of all hyperplane solution sets, and the three vector $\mathbf{x}^{(k)} - \mathbf{x}^*$, $\mathbf{x}^{(k-1)} - \mathbf{x}^*$, and $\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}$ form the edges of a right triangle. By the Pythagorean theorem, we have

$$\|\mathbf{x}^{(k-1)} - \mathbf{x}^*\|^2 = \|\mathbf{x}^{(k)} - \mathbf{x}^*\|^2 + \|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|^2.$$

The result follows since $\|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|^2 \geq 0$.